

TechScan – Exploring the use of multi-agent AI for tech news scanning and literature review (Prototype1)

Elizabeth Lim

Log: 01/09/2025

1. Introduction

Staying informed about the latest advancements in science and technology is important, as modern research often involves multiple disciplines. However, it can be time-consuming and challenging to delve into articles and news in unfamiliar fields to extract key information.

Often in cross discipline research projects, a researcher just wants to gain a quick overview of a topic, only to become overwhelmed by the sheer volume of available information if the subject is currently popular or heavily researched by many.

A possible solution could be the use of multi-agent AI for such a task. Such an AI could autonomously search the internet for the most recent developments on a specific topic, analyse emerging trends, and generate concise summaries of its findings. This would provide users with a valuable, concise and quick overview which could be beneficial for guiding research directions and informing decision-making. Google's Gemini Flash and OpenAI ChatGPT are capable of performing research assistance. The advantages of having a smaller scale version that can work locally would be security, confidentiality and lower cost.

Local research agentic systems could provide a starting point for further topic exploration, initial topic sensing and suggest initial explorations.

2. Objectives

- i) Prototype an agentic AI system that gathers information, news, product press releases and research can perform generates a summary report of key findings for research direction advisory.
- ii) Explore the requirements for a competent multi-agent AI crew for performing tech discovery and literature review.

3. Approach

- i) Build an agentic AI system using Python with crewAI.
- ii) Enable access to online searches through API calls.
- iii) Define required agent roles, tools and tasks.
- iv) Evaluate performance based on logs and outputs.

4. Methods

4.1. Overview of the system

The workflow for TechScan (Figure 1) would begin with user specifying a topic. There are three main tasks:

1. A search on the internet would be performed to gather information which would include, news, research publications, press releases, product releases on the topic.
2. The information gathered from the websearch would have to be collated, analysed and evaluated for relevance and importance. It would be beneficial if trends can be identified.
3. A summary report (of no more than 3 pages) would be generated which would include what was found by the agents, and recommended direction for research and development.

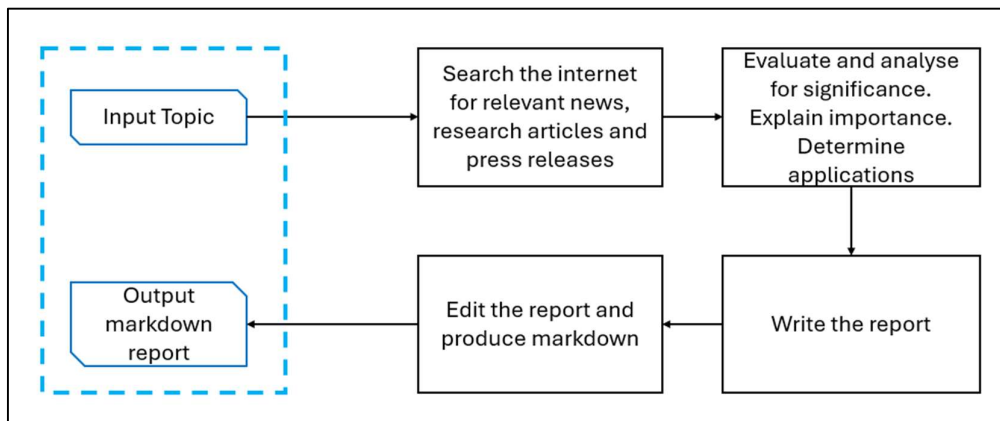


Figure 1: TechScan Basic Workflow

4.2. Development Platform

VSCode is the platform of choice. Using Poetry (<https://python-poetry.org/>), a python 3.11 environment was created containing `crewai` (<https://www.crewai.com/>) and `crewai-tools` (<https://docs.crewai.com/en/tools/overview>) modules.

CrewAI was selected over Langchain/Langgraph (<https://www.langchain.com/langgraph>) for multi-agent AI development for this project because it seem to present as a simple and straightforward way of defining agents, tools and tasks, and orchestrating them to achieve the goal.

For access to google search, Serper (<https://serper.dev/>) provided 2500 free requests for Google search call upon registration.

The LLM is a local LLM installed and accessed through LMStudio. The model used was Gemma-3n-E4B-it-text-GGUF. This small model was chosen as it would run reasonably well on a gaming laptop. In addition, a local LLM would not be subject to

subscription costs. A larger model could be accessed via Open Router but there would be a limit on usage on free subscription.

Python scripts were written and ran in a notebook type interface (Jupyter Notebook or Ipython)

4.3. Agent, Tool and Task Definitions

This system was made up of a crew of four agents, each with a task:

- i) **Tech News Researcher** would be tasked to search for information, news and publications on the topic, and create a list of its findings.
- ii) **Tech Consultant / Advisor** would look through the list generated by the Researcher, perform a deep dive into the topics and come with a conclusion or recommendation.
- iii) **Tech Writer** would collate the findings of the Researcher and Advisor, and writes a concise summary report.
- iv) **Tech Report Editor** would edit the report for grammatical errors and ensure that the report doesn't contain inconsistencies.

Under agent definitions, the goal gives the agent an objective and its main motivation. The backstory of a agent guides the agent's behaviour and 'thought process'. (Table 1)

Table 1: Agent roles, goals and back stories.

Role	Goal	Backstory
Tech News Researcher	Collect the latest news, product releases and articles on latest research breakthroughs and findings on {topic}.	You are a diligent and resourceful tech news researcher. You have access to the internet and can use tools to gather information. Your goal is to find the most recent and relevant news articles, product releases, and research breakthroughs related to the {topic}. You are gathering information for a report on the latest developments in {topic}.
Tech Consultant	Evaluates and analyses the findings gathered by the Tech News Researcher, determines the significance and general trends and developments to highlight with regards to {topic}	You are an expert technologist with a background in industry and applied research and development. You are evaluating the findings gathered by the Tech News Researcher to determine the most significant trends and developments in {topic}. You will provide insights and recommendations based on the latest news, product releases, and research breakthroughs.
Tech Writer	write a summary report of the key findings about {topic}	You are skilled in science communications and science writing. You are working on producing a report of no more than three pages summarizing the findings of the Tech

		News Researcher and the analysis and evaluation provided by Tech Consultant. Your audience are scientists and researchers interested in latest developments and products/equipment/technologies for {topic}. You will write in clear, concise language and avoid jargon.
Tech Editor	Ensures that the final report is in a clear and consistent format, without grammatical errors and contradictions.	You are skilled in editing and proofreading. You are working on ensuring that the final report is in a clear and consistent format, without grammatical errors and contradictions. You will ensure that the report is well-structured and easy to read. You will also ensure that the report is free of any inconsistencies.

For simplicity, the agents were initially equipped with two tools:

- i) A search tool using crewai's `SerperDevTool(delay=1, n_results=5)`. This allows the agent to search the web to look for and verify information.
- ii) A website scraping tool using `ScrapeWebsiteTool()`. This enables the agent to obtain information from the website.

The four tasks for the four agents are defined with more details, directing what outputs are required (Table 2). The task output defines the expected output from the agent, either a json object, a text file or markdown text file. Outputs enable one to evaluate the performance of the agent. Task description contains details on how the task is to be accomplished.

Table 2: Agent and task descriptions

Agent	Task Output	Task Description
Tech News Researcher	A clean JSON object with tech details following the exact schema provided. Do not include backticks or comments.	"Search for news, articles, product releases and research papers related to {topic} for {application} starting {from_year}. " "Summarize the key findings in a concise manner. "
Tech Consultant	"A detailed analysis of the key trends and developments in {topic} for {application}, including their significance and potential impact."	"Evaluate and analyse the findings gathered by the Tech News Researcher on {topic}. " "Determine the significance and general trends and developments to highlight. " "Provide insights and recommendations based on the latest news, product releases, and research breakthroughs."

Tech Writer	A well-written and concise report summarizing the key findings about {topic}.	"Write a summary report of the key findings about {topic} for {application}. " "The report is based on the findings of the Tech News Researcher and the analysis provided by Tech Consultant. " "The audience is a general audience with an interest in technology but not necessarily a technical background. " "Write in clear, concise language and avoid jargon. " "Highlight the most important and interesting developments in {topic}."
Tech Editor	A polished and error-free report in markdown that is clear, consistent, and easy to read.	"Ensure that the final report is in a clear and consistent format, without grammatical errors and contradictions. " "Ensure that the report is well-structured and easy to read. " "Ensure that the report is free of any factual errors or inconsistencies."

All agents were given access to the two tools and set with the ability to delegate (`allow_delegation=False`).

4.4. User Input

The input would specify a topic (or technology) and an application for that technology. The feature 'from_year' ensures the model does not search too far back for information.

```
# Example data for kicking off the process
Techscan_crew_inputs = {
    'topic': 'genomics',
    'application': 'cancer detection',
    'from_year': '2020',
}
```

Box 1: User input specifications

Hierarchical process was used with setting `max_iteration=2` to limit the times the agents run through the workflow (as they try to improve on their findings and verify them). Verbose is set to `True` so that it is possible to monitor the agents during the run and see agent thought processes. (Box 2)

```
from crewai import Crew, Process

# Define the crew with agents and tasks
techscan_crew = Crew(
    agents=[tech_news_agent,
            tech_consultant_agent,
            tech_writer_agent,
            tech_editor_agent],

    tasks=[gather_info_task,
           analysis_task,
           write_report_task,
           edit_report_task],

    manager_llm=llm,
    process=Process.hierarchical,
    max_iterations=2,
    verbose=True
)
```

Box 2: Process specifications, agent order, task order and process type.

5. Results and Observations

5.1. Test run results and iterative changes to the initial system

Changes were made to the agentic AI system based on observing test runs (verbose = True) and examining the generated files. Most runs completed under an hour.

It was observed that the steps and time taken to generate the report varied from run to run even when no changes were made. The quality and content of the report also varied – sometimes only a single paragraph is generated and sometimes a full summary report with expected format of sections and paragraphs were generated.

Most of the runs did not complete due to various reasons, demonstrating the limit of the local LLM, the random/fuzzy nature of generative AI and the importance of writing explicit and clear instructional prompts. (Table 3).

Table 3: Issue log and comments

Problems	Possible causes and solutions	Future improvement
The LLM agent would save the json files with backticks and comment at times, which results in error and stops the run.	Tried explicit instructions in the prompt. While json format would allow better machine processing, the LLM is not able to consistently save information in this format. One solution might be to write a program that save the information as strings in a list of list or dictionary before passing it to another function to turn it into a json object.	Better prompting. Consider other ways to save gathered information. Don't depend on LLM to correctly save info in json format.

Agent not saving search results in expected format or not all search results saved.	This second issue with saving to json file may be caused by an error in the Pydantic class definition.	Explicit instruction to LLM to save all results.
Writer Agent / Editor Agent stuck in endless loop of refining results. Writer Agent not generating report according to previous agent's gathered information	This issue could be caused by upstream issue of information not being saved properly by researcher and consultant agents, and that the Writer and Editor Agents were not given adequate tools to read the files generated by the previous agents. Thus, these last two agents are left to regenerate information based on their own web searches. When the tools for web search and web scraping were taken away from these two agents in the last run, the final report seemed to present the information found by the Researcher and Consultant agents. There are suggestions that these two agents may require a <code>ReadFileTool</code>. if the agents should be referencing gathered data. Interestingly, the addition of this tool resulted in very short reports.	Give writer and editor agents ability to read log files of search agents, however, instructions should be given as to how it should use this. Do not allow writer and editor agents access to the internet.
Final output report information loss due to Crew Manager capability	In hierarchical process, crewai creates a default crew manager agent based on the LLM used to help manage and coordinate the work of the agents. It was suggested that crew manager should be an LLM trained for high reasoning and could be a different LLM from the other agents.	A different or more powerful LLM for the role of crew manager.
Search results are inconsistent and not reproducible and lacks source reference.	Inconsistent findings and a non-reproducible final report seem to suggest that human input or more explicit instructions may be required in the initial prompt to guide the search or make the search more systematic. Perhaps more context may be required.	Human input requirement. Consider a different search algorithm and not one dependent or entirely dependent on LLM.

5.2. Factors affecting the quality of the generated report

The generated report contains general information and does not delve deeper into each found search object. It is likely website are blocking Serper API from accessing these articles. The other contributing factor to the shallowness of the report is that the agents do not 'read' the articles in the way that a human would read and understand the articles.

On this point, it may be that LLM ability to search for articles could be improved through better instruction, however the articles might require a human to read and evaluate for inclusion into the final report. It probably isn't realistic to expect an LLM (and a local small LLM) to be able to 'read' articles by itself. Even with additional tools and API to enable the LLM to access the articles itself, LLMs may summarize and interpret the articles wrongly,

LLM are fundamentally autocomplete machines and work best at predicting sequences and patterns. (Syed, 2024; <https://blog.bboxcars.ai/p/beyond-autocomplete-guiding-llms>) LLMs are not guaranteed to give factual and accurate outputs. It may be possible that given the right context and prompt, it could get close to being accurate and perhaps exhibit 'understanding' on the given topics and data resources. (Steyvers et al, 2025) It is also possible that using a more advance and larger LLM may also improve results.

5.3. Context engineering

Context engineering techniques may be required to manage context size. LLM performance is known to deteriorate due to long context issues (bloating, accumulation of irrelevant or conflicting information etc, see: <https://www.dbreunig.com/2025/06/22/how-contexts-fail-and-how-to-fix-them.html>) This would explain how the initial rounds produced results, yet the agent seemed to delete or cut down its own work in subsequent rounds.

5.4. Safe and respectful search request rates required

It would be highly recommended **to implement safe and respectful searching** using websearch and webscraper tools. Many websites would block bots from searching and scraping, and those that allow it would have rate restrictions where sending too many requests may result in getting one blocked or tagged as suspicious. This could be a contributing factor to why the LLM agents fail in their task as they may get blocked when they make repeated requests.

6. Conclusions

6.1. Final system setup and run

The final system set up consisted of four agents with search, analysis, writing and editing roles respectively. The agentic AI crew was set to run via hierarchical process with maximum of two iterations for a reasonable turnaround time, and all agents produced output files upon completion of their assigned task.

In the final setup, two final reports in markdown were successfully generated, and they varied in layout style and content. The layout could be more precisely defined with better instructions, or examples in the prompt. Again, the content reflects the randomness of the LLM.

6.2. Recommendations moving forward (if building one's own system)

1. Rethink on how better guide the LLM in article and journal search
2. Use a different search engine.
3. Explore the use of other python libraries for writing Agentic AI systems
4. Better instructions and prompts required which would include guardrails against hallucinations, requiring real source attribution and examples for required outputs.
5. Consider including human input into the workflow.
6. Explore the use of small LLMs (SLM) for AI agent roles.
7. Consider using single agent rather than a multi-agent system.

7. References

1. Steyvers, M., Tejada, H., Kumar, A. *et al.* What large language models know and what people think they know. *Nat Mach Intell* **7**, 221–231 (2025).
<https://doi.org/10.1038/s42256-024-00976-7>
2. A tutorial showing how to build a research agent system using Langgraph
<https://towardsdatascience.com/langgraph-101-lets-build-a-deep-research-agent/>